

# Course Outline: Metrics, Aggregation, and Grouping

---

**Title:** Metrics, Aggregation, and Grouping: The Formula Behind Every Report **Skill:** Skill 44: Building a report from the telemetry or user data **Learnpack:** + Agregación, agrupamiento y métrica **File:** s44-w15d44-agregacion-agrupamiento-y-metrica **Prerequisite:** s44-w15d44-de-datos-crudos-a-reportes-transformando **Target Audience:** Beginner developers who have learned the transformation pipeline and now want to master the mental model behind data aggregation **Total Lessons:** 8 **Format:** Mixed (12% hands-on = 1 lesson)

---

## Course Description

You've run the transformation pipeline. Now understand why it works. A metric is a question you want to answer — "completions per day", "average sessions per week", "total revenue per user". Every metric follows the same formula: METRIC = AGGREGATION(column) grouped by DIMENSION. This course ingrains that formula through the full range of aggregation functions (.count(), .sum(), .mean(), .min(), .max(), .agg()), shows how grouping dimensions change what a number means, and connects it all to SQL GROUP BY and dashboard tools you'll encounter in the wild.

---

## Course Philosophy

This course emphasizes:

- The mental model first, the code second
  - The distinction between a global number and a contextual number
  - Recognizing the same concept (aggregation + dimension) across tools — pandas, SQL, dashboards
- 

## Course Statistics Summary

| Section                    | Lessons  |
|----------------------------|----------|
| 00 - Welcome               | 1        |
| 01 - The Metric Formula    | 2        |
| 02 - Aggregation Functions | 3        |
| 03 - Assessment            | 1        |
| 04 - Outro                 | 1        |
| <b>Total</b>               | <b>8</b> |

---

## Learning Objectives:

- Define what a metric is in the context of data analysis
- Understand why aggregation and grouping must be combined to produce useful metrics
- Preview the course structure

## Content Outline:

- A metric is not raw data — it's an answer to a question
- Examples: "completions per day", "average time-to-close per user", "total weekly revenue"
- The formula that produces every metric:  $\text{METRIC} = \text{AGGREGATION} + \text{DIMENSION}$
- Why a single aggregate number (e.g., total count = 45) is usually less useful than the same number segmented by a dimension
- Course overview: Sections 01 and 02 teach the formula; Section 02 ends with the hands-on challenge

**Transitions:** This lesson frames the problem. Section 01 names the formula precisely.

## Key Principles:

- A metric without context is just a number — grouping adds the context
- The same aggregation function applied to different dimensions produces different insights
- Metrics are not discovered in raw data; they are computed from it

## Examples:

- `df['id'].count()` → 45 (how many TODOs exist — mostly useless)
  - `df.groupby('date')['id'].count()` → {2024-01-15: 7, 2024-01-16: 3, ...} (completions per day — actionable)
  - The second version answers a real question; the first answers nothing useful
- 

## 01.0 The Metric Formula

### Learning Objectives:

- State the mental formula:  $\text{METRIC} = \text{AGGREGATION}(\text{column}) \text{ grouped by } \text{DIMENSION}$
- Identify the three components of a metric: the question, the aggregation, and the dimension
- Map real-world metric names to their formula components

### Content Outline:

- What is a metric? A metric is the question you want answered expressed as a number
- Examples of metrics and their questions:
  - "Completions per day" → COUNT of completions grouped by date
  - "Daily average" → MEAN of daily counts (chained: first group by day, then average the series)
  - "Total weekly revenue" → SUM of total\_price grouped by week
- Decomposing each metric into:  $\text{AGGREGATION FUNCTION} + \text{COLUMN} + \text{DIMENSION}$
- The formula expressed in code: `df.groupby(DIMENSION)[COLUMN].AGGREGATION()`

- How the formula maps to SQL: `SELECT date, COUNT(id) FROM todos GROUP BY date`
- Connection to dashboard tools: the same formula runs inside Metabase, Tableau, and BigQuery — pandas is just the code version
- Anti-pattern: choosing the aggregation function before defining the metric question — this produces code that runs but answers nothing

**Transitions:** The formula is the concept. The next lesson (01.1) shows how different dimensions change the meaning of the answer.

### Key Principles:

- Define the metric question first: "What do I want to know?" → then select the aggregation and dimension
- The same column can produce completely different metrics depending on the dimension
- The SQL analogy makes the formula portable: learn it once, use it everywhere

### Examples:

- Question: "How many TODOs are completed each day?" → `df.groupby(df['completed_at'].dt.date)['id'].count()`
  - Question: "What is the average daily completion count this week?" → `daily.mean()` (where daily is the previous series)
  - SQL: `SELECT WEEK(created_at), SUM(total_price) FROM orders GROUP BY WEEK(created_at)`
- 

## 01.1 Grouping by Dimension

### Learning Objectives:

- Explain the difference between a global aggregate and a segmented aggregate
- Apply different grouping dimensions (day, user, category, status) to the same dataset
- Recognize when a metric is too coarse and needs a more specific dimension

### Content Outline:

- Grouping = segmenting the data by a shared characteristic before aggregating
- Common grouping dimensions:
  - By time: day (`dt.date`), week (`dt.isocalendar().week`), month (`dt.month`)
  - By entity: user, category, product, region
  - By status: completed, pending, active
- The contrast: global count vs segmented count
  - `df['status'].count() = 45` (tells you nothing about patterns)
  - `df.groupby('date')['status'].count()` = shows the trend over time
  - `df.groupby('user_id')['id'].count()` = shows which users are most active
- Multi-dimensional grouping: `df.groupby(['user_id', df['completed_at'].dt.date])['id'].count()` — shows each user's daily activity
- When to add dimensions vs when to keep it simple: more dimensions = more specific, but also smaller numbers per bucket

- Anti-pattern: grouping by a high-cardinality dimension (e.g., individual timestamps) — produces a unique row per event, no aggregation happens

**Transitions:** You now understand how grouping shapes the metric. Section 02 teaches the aggregation functions that compute the values within each group.

### Key Principles:

- Context comes from the dimension, not from the aggregation function
- Adding a dimension never changes the aggregation function — it changes what each row in the result represents
- A good grouping dimension has meaningful, bounded categories (days, users, categories) — not unbounded unique values

### Examples:

```
# Global – no context
df['id'].count() # → 45

# Segmented by date – daily trend
df.groupby(df['completed_at'].dt.date)['id'].count()
# → {2024-01-15: 7, 2024-01-16: 3, 2024-01-17: 12}

# Segmented by user – per-user activity
df.groupby('user_id')['id'].count()
# → {user_1: 15, user_2: 8, user_3: 22}
```

---

## 02.0 Aggregation Functions

### Learning Objectives:

- Name the core aggregation functions in pandas and their purposes
- Choose the correct function based on what the metric is measuring
- Understand that aggregation collapses many rows into one value per group

### Content Outline:

- What aggregation does: takes all rows in a group and reduces them to one value
- The core functions:
  - `.count()` — how many rows? Use for event counts, completion counts
  - `.sum()` — what is the total? Use for revenue, duration, quantity
  - `.mean()` — what is the average? Use for rates, averages, performance benchmarks
  - `.min()` / `.max()` — what are the extremes? Use for first event, highest value, range
- Decision guide: match the function to the metric question
  - "How many?" → `.count()`
  - "What is the total?" → `.sum()`
  - "What is the average?" → `.mean()`

- "When did it first/last happen?" → `.min()` / `.max()`
- The critical rule: always specify the column after `groupby`, not the full grouped DataFrame
  - `grouped['id'].count()` is correct — counts only the 'id' column
  - `grouped.count()` counts ALL columns, including those with NaN values, and produces misleading results
- Section preview: 02.1 covers `.agg()` for computing multiple metrics in one pass; 02.2 is the hands-on challenge

**Transitions:** You can now select the right function for any metric. 02.1 shows how to compute several metrics at once.

### Key Principles:

- Always know what you're aggregating: the column matters as much as the function
- `.count()` counts non-null values per column — specifying the column keeps this predictable
- The choice of aggregation function is the choice of what the metric means

### Examples:

```
# Count: how many completed TODOs per day?
df.groupby(df['completed_at'].dt.date)['id'].count()

# Sum: total revenue per week?
df.groupby(df['created_at'].dt.isocalendar().week)['total_price'].sum()

# Mean: average session duration per user?
df.groupby('user_id')['duration_minutes'].mean()
```

## 02.1 Multi-Aggregation with `.agg()`

### Learning Objectives:

- Use `.agg()` to compute multiple metrics for the same group in one operation
- Pass a dictionary to `.agg()` to specify different functions for different columns
- Read and interpret a multi-column aggregation result

### Content Outline:

- The limitation of single-function aggregation: you need one chain per metric
- `.agg()` as the solution: compute all metrics in a single grouped operation
- Basic usage: `grouped.agg('count')` — equivalent to `.count()`
- Dict syntax: `grouped.agg({'col1': 'count', 'col2': 'mean'})` — different functions per column
- Named aggregation (pandas ≥ 0.25): `grouped.agg(metric_name=('col', 'func'))` — result has readable column names
- Chained operations: `df.groupby('date')['id'].count().mean()` — first group-count per day, then average those daily counts to get the "average daily completions" figure

- The formula for average daily completions: `df.groupby('date')['id'].count().mean()` = daily series → average of that series
- Connection to the mental formula: METRIC = AGGREGATION(column) agrupada por DIMENSION; chaining adds a second aggregation level
- Anti-pattern: calling `.agg('count')` without specifying the column — returns count for all columns including NaN-affected ones

**Transitions:** You have the full toolkit. 02.2 applies it in a hands-on challenge that produces real output.

### Key Principles:

- `.agg()` is the idiomatic pandas way to compute multiple metrics at once — prefer it over multiple separate groupby calls on the same data
- Named aggregation makes results readable without renaming columns afterward
- Chained aggregation (e.g., daily counts → average) is a valid pattern for computing "averages of aggregates"

### Examples:

```
# Named aggregation: total and completed count per user
by_user = df.groupby('user_id').agg(
    total=('id', 'count'),
    completed=('status', lambda x: (x == 'completed').sum())
)

# Average daily completions (chained)
daily_avg = df[df['status'] == 'completed'] \
    .groupby(df['completed_at'].dt.date)['id'] \
    .count() \
    .mean()

# → e.g., 8.4 completions per day on average
```

## 02.2 Apply the Formula

**Challenge Prompt:** Given a CSV file of TODO events with columns `id`, `user_id`, `status`, and `completed_at`, write a Python script that computes three metrics: (1) daily completion counts for the last 7 days, (2) the daily average from those counts, and (3) completion rate per user (completed ÷ total) using `.agg()`.

### Solution Code:

```
import pandas as pd
import json
from datetime import datetime, timedelta

df = pd.read_csv('todos.csv')
df['completed_at'] = pd.to_datetime(df['completed_at'])
```

```

cutoff = datetime.now() - timedelta(days=7)
recent = df[df['completed_at'] >= cutoff]

daily = recent[recent['status'] == 'completed'] \
    .groupby(recent['completed_at'].dt.date)['id'].count()

daily_avg = round(daily.mean(), 2)

by_user = df.groupby('user_id').agg(
    total=('id', 'count'),
    completed=('status', lambda x: (x == 'completed').sum())
)
by_user['rate'] = (by_user['completed'] / by_user['total']).round(2)

print("Daily completions:", json.dumps({str(k): int(v) for k, v in
daily.items()}), indent=2))
print("Daily average:", daily_avg)
print(by_user[['total', 'completed', 'rate']].to_string())

```

### Challenge Code:

```

import pandas as pd
import json
from datetime import datetime, timedelta

df = pd.read_csv('todos.csv')

# Convert 'completed_at' to datetime
# your code here

# Define cutoff: 7 days ago
# your code here

# Filter rows within the last 7 days
# your code here

# Compute daily completion counts (filter completed, group by day, count)
# your code here

# Compute the daily average from the daily counts
# your code here

# Compute total and completed counts per user, then calculate rate
# use .agg() with named aggregations
# your code here

# Print daily completions as JSON, daily average, and by-user table
# your code here

```

---

## 03.0 Assessment 🧠

### Learning Objectives:

- Apply the METRIC = AGGREGATION + DIMENSION formula to new scenarios
- Identify the correct aggregation function for each metric type
- Distinguish contextual metrics from global aggregates

### Question Format: Scenario-based (multiple-choice)

#### Questions:

1. A metric is best described as:

- A) A raw column in a DataFrame
- B) A question about data, expressed as a number produced by aggregation over a grouped dimension ✓
- C) A filtered subset of rows
- D) The result of loading a CSV file

2. You want to know the total revenue per week from an orders table. The correct formula is:

- A) `df['total_price'].sum()`
- B) `df.groupby('user_id')['total_price'].mean()`
- C) `df.groupby(df['created_at'].dt.isocalendar().week)['total_price'].sum()` ✓
- D) `df.groupby('total_price')['id'].count()`

3. `df['id'].count()` returns 45. Why is this result usually less useful than `df.groupby('date')['id'].count()`?

- A) `.count()` is incorrect syntax without `groupby`
- B) The global count gives no information about patterns over time; the grouped version shows the daily trend ✓
- C) `count()` only works on string columns
- D) 45 is always incorrect

4. Which aggregation function should you use to answer "What is the highest total\_price in each user's order history?"

- A) `.count()`
- B) `.sum()`
- C) `.mean()`
- D) `.max()` ✓

5. You call `df.groupby('user_id').agg({'id': 'count', 'duration_minutes': 'mean'})`. What does the result contain?

- A) One row per TODO with both columns computed globally



- B) One row per user\_id with the count of ids and the mean duration ✓
- C) One row with the total count and total mean across all users
- D) An error because you can't mix count and mean in one agg()

6. `df.groupby('date')['id'].count().mean()` computes:

- A) The total count of all ids
- B) The average id value
- C) The average of the daily completion counts ✓
- D) A syntax error

7. You need to compute both total todos and completed todos per user. The most efficient approach is:

- A) Two separate groupby calls, one for total and one for completed
- B) One `.agg()` call with named aggregations for both metrics ✓
- C) Filter the DataFrame twice, then concat the results
- D) Use `.sum()` on the status column directly

8. The SQL equivalent of `df.groupby('category')['revenue'].sum()` is:

- A) `SELECT SUM(revenue) FROM orders`
- B) `SELECT revenue, COUNT(*) FROM orders GROUP BY category`
- C) `SELECT category, SUM(revenue) FROM orders GROUP BY category` ✓
- D) `SELECT category FROM orders WHERE revenue > 0`

### Evaluation Criteria:

- Correctly applies the METRIC = AGGREGATION + DIMENSION formula
- Selects the appropriate aggregation function for each metric type
- Understands the difference between global and segmented aggregation
- Recognizes `.agg()` as the idiomatic multi-metric tool

### Score Interpretation:

- 8/8: The formula is yours — apply it to any dataset
- 6–7: Review any missed lessons before moving on
- 4–5: Re-read Sections 01 and 02 with focus on the formula decomposition
- Below 4: Restart from Section 01

## 04.0 Metrics in Practice

### Content Outline:

- Course recap: what students accomplished
  - Learned the mental formula: METRIC = AGGREGATION(column) grouped by DIMENSION
  - Understood how different dimensions (day, user, category) change what a number means
  - Practiced all core aggregation functions (`.count()`, `.sum()`, `.mean()`, `.min()`, `.max()`, `.agg()`)
  - Used `.agg()` for multi-metric computation in one pass

- Connected pandas aggregation to SQL GROUP BY and dashboard tools
- Connection to bigger picture: every analytics feature — dashboards, product metrics, ML feature engineering, A/B test results — uses this formula. The tool changes (SQL, pandas, Spark, BigQuery), the formula doesn't.
- Next steps: Skill 45 shows how to schedule the pipeline to run on a recurring basis; the combination of transformation + scheduling = automated metrics that update themselves
- Resources for further exploration:
  - pandas `.agg()` documentation: named aggregation patterns
  - "Python for Data Analysis" by Wes McKinney — the pandas textbook
  - Practice: take any dataset you have, define 3 metrics as questions, then write each as a groupby + agg formula
- Encouragement: the formula seems simple because it is — METRIC = AGGREGATION + DIMENSION. That simplicity is the point. Complex analytics tools at Spotify, Netflix, and Google all implement this same underlying pattern at scale.

**Note:** This is conclusion only — no theory, exercises, or assessment.